Beyond Pass/Fail: Golden-Model Verification of MBIST and Self-Repair Logic

Abhinav Kidambi, IEEE Member
Independent Researcher, USA
abhinavkidambi1989@gmail.com

***Abstract*—Memory built-in self-test (MBIST) and built-in self-repair (BISR) logic decide whether a defective memory is shipped, repaired or discarded, yet their RTL is commonly verified only by checking that the test completes and that its pass/fail flag looks plausible. We present a golden-model verification method that checks every observable MBIST/BISR result (the pass/fail flag, each reported fail address and the final repair table) against an operation-level March reference model under systematic memory fault injection, and qualifies the checker itself with seeded RTL bugs. On an open-source MBIST/BISR design taped out on a public shuttle, 4,032 injected stuck-at, transition and coupling faults show that its completion-only testbench exposes no bug and a pass/fail check exposes only 2.8% of detectable faults (72 escapes, 95% CI 2.3–3.6%), whereas the full scoreboard exposes bugs in 100% of them: every fail address and repair entry is wrong and the faulty cell is repaired in only 1.6% of failing runs. On a second open-source MBIST with row-redundancy repair the same scoreboard raises no false alarm in 1,694 runs (95% upper bound 0.23%). Across 34 seeded bugs, completion-only checking kills 15%, pass/fail checking 62% and the full scoreboard 79%. The flow uses only open-source tools and runs in seconds to minutes.**

***Index Terms*—MBIST, built-in self-repair, March test, memory fault models, fault injection, mutation analysis, design verification**

I. Introduction

Embedded memories dominate the area of modern SoCs, and nearly every memory instance is wrapped in MBIST and, increasingly, repair logic [1]. A bug in this logic is costly in both directions: a missed fault ships a defective part, while a wrong fail address or repair entry wastes redundancy or "repairs" a good cell and leaves the bad one in use. Because MBIST runs on every die at every insertion, such a bug is replicated across the whole product.

In practice MBIST RTL is often verified by running the test on a fault-free memory and checking that DONE rises and FAIL stays low, sometimes with a few injected faults checked for FAIL. Such checks say little about the information that repair and diagnosis consume: *which* addresses failed, and *what* was written into the repair registers. They also say nothing about how many real bugs the checks themselves would miss.

This paper contributes (a) a reference-model-driven method that predicts, for any injected fault, the pass/fail flag, the ordered fail-address list and the repair table, and scores the RTL at three increasing checker levels (Section III); (b) an open-source fault-injection and seeded-bug campaign flow with confidence intervals for every rate (Section IV); (c) a case study on a taped-out MBIST/BISR design in which the method finds escapes, wrong fail addresses and a broken repair table that the design's own testbench misses, with register-level root causes (Sections V-A–V-C); (d) a control study on a second design in which the same scoreboard raises no false alarm, and a seeded-bug qualification of all checker levels on both designs (Sections V-D–V-E); and (e) design recommendations for MBIST/BISR logic (Section VI).

II. Background and Related Work

March tests. A March test applies March elements, each a series of reads and writes applied to every address in ascending ($\Uparrow$), descending ($\Downarrow$) or either ($\Updownarrow$) order [2], [3]. MATS+ [2], [4] detects stuck-at faults (SAF) in $5N$ operations; March X ($6N$) and March Y ($8N$) add transition faults (TF); March C– [3], [5], $\{\Updownarrow\ (w0); \Uparrow\ (r0, w1); \Uparrow\ (r1, w0); \Downarrow\ (r0, w1); \Downarrow\ (r1, w0); \Updownarrow\ (r0)\}$, detects all SAFs, TFs, address-decoder faults and unlinked inter-word idempotent coupling faults (CF) in $10N$. Intra-word CFs need several data backgrounds [2]; linked faults need longer tests [6], [7].

Verifying memory tests and MBIST. Van de Goor and Smit automated the verification of March *algorithms* against fault models [8]; that work does not check the RTL that implements an algorithm. Assertion-based verification has been applied to March-based MBIST controllers [9], and a recent industrial methodology closes coverage gaps in shared-bus MBIST with physical-design-aware checks [10].

Repair and testbench quality. BISR stores failing addresses and remaps accesses to spare rows, columns or words [1], [11], so its correctness depends directly on the fail addresses MBIST reports. Mutation analysis [12], [13] measures how many seeded bugs a test environment detects.

We differ from these works in checking, against a golden model, *every* result an MBIST/BISR block produces under a systematic fault campaign, in quantifying what weaker checkers miss with both injected faults and seeded RTL bugs, and in being reproducible with open designs and tools.

III. Method

Fig. 1 shows the flow. For each fault, the testbench injects it into the behavioural memory of the unmodified RTL, runs MBIST to completion and records every fail report and the final repair table. Independently, the golden model applies the specified algorithm to an operation-level memory with the same fault and predicts the correct results. A scoreboard

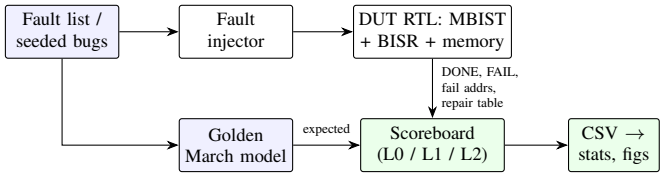


Fig. 1. Verification flow. Blue: inputs and golden reference; white: unmodified RTL with behavioural fault injection; green: scoring and statistics.

compares the two at three checker levels and writes one row per run.

A. Golden model

The golden model simulates a $D \times W$ memory at the operation level. An algorithm is a list of (order, operations) pairs, optionally repeated for a list of data backgrounds, with data '0' the background and '1' its complement. Each read compares the whole word; a mismatching read appends its address to the expected fail list. The expected repair table is the first R distinct failing addresses in detection order. Spare words are assumed fault-free and outside the tested range, the documented intent of both designs. The model reproduces March theory on our fault list (Fig. 10): MATS+ detects 50% of TFs, March X and March Y 50% of inter-word CFs, March C– all of them, and four data backgrounds all intra-word CFs.

B. Fault injection

Faults are injected into the RTL memory array without editing the design. A correction is applied just after every clock edge, after any write has landed and before the next read samples the array: an SAF forces the cell; a TF reverts a forbidden $0 \to 1$ or $1 \to 0$ transition; an idempotent CF forces the victim when the aggressor makes a given transition. Up to eight simultaneous faults are supported. The golden model uses the same semantics.

C. Checker levels

Each run is scored at increasing levels: **L0**, DONE asserted (completion only); **L1**, DONE and the pass/fail flag equal the prediction; **L1c**, L1 plus the CORRECT flag and error count where the design exposes them (what Design B's own testbench checks); **L2**, the full scoreboard, adding every fail address and the repair table. A run *exposes a design bug* at a level if that level's comparison fails.

D. Seeded bugs and statistics

To qualify the checkers themselves, each seeded bug is a one-line RTL change (an operator, constant, index or condition) applied to a copy of the sources. A seeded bug is *killed* at a level if, on some test, the outputs visible at that level differ from the unmodified design's. Rates are reported with Wilson 95% intervals [14]; for zero events the upper bound is given.

TABLE I
DESIGNS AND CAMPAIGN SIZES.

	Design A (TT #777)	Design B (mbist_ctrl)
Memory / spares	32×8 / 2 rows	512×32 / 4 words
Algorithm	March X, solid data	5 el. $\times$ 8 backgrounds
Fault runs	4,032	1,694
Seeded bugs $\times$ tests	16×129	18×7
Cycles per MBIST run*	228	162,563
Simulation time per run	0.01 s	2.05 s

*Fault-free run, from start to DONE; Icarus Verilog 14.0 (devel).

IV. Designs and Experimental Setup

Design A (Fig. 2a) is Tiny Tapeout IHP 26a project #777 [15], an MBIST with a CAM-style BISR, at its taped-out commit (Apache-2.0). The RTL implements a 32×8 memory with two repair entries remapped to rows 30–31, which are also ordinary tested addresses; the documentation describes a 256×8 memory with 16 entries. Although documented as March C–, the controller implements March X, $\{\Uparrow (w0); \Uparrow (r0, w1); \Downarrow (r1, w0); \Uparrow (r0)\}$.

Design B (Fig. 2b) is mbist_ctrl [16] (Apache-2.0), taped out on an Efabless/SkyWater shuttle. It tests a 512×32 two-port SRAM with a programmable controller; by default eight data backgrounds $\times$ five March elements with read-after-write in both directions, over words 0x000–0x1FB, with four spare words 0x1FC–0x1FF and an error limit of four. We simulate the MBIST core with the SkyWater SRAM behavioural model wired as on the chip, with standard cells replaced by behavioural equivalents. Its golden model works at the repair level: for a set W of faulty user words a correct design reports ERROR = 0, CORRECT = 1, count $|W|$ and repair table W if $|W| \leq 4$, and ERROR = 1 otherwise.

Campaigns (Table I). Design A: SAF0/1 and TF↑↓ in all 256 cells and idempotent CFs with physically adjacent aggressors (row above, row below, neighbouring bit), both transitions and forced values. Design B: an SAF in every user word, a TF in every second word, 30 random sets each of 2–6 faulty words and 20 faulty-spare cases. Seeded bugs: 16 in Design A (controller and BISR) and 18 in Design B (address generator, comparator, repair, FSM, pattern and stimulus selection).

Tools. Icarus Verilog 14.0 [17]; Python for the golden model, campaigns, statistics and plots. The directed tests behind Table III were also reproduced on the EDA Playground web service with Icarus Verilog 12.0.

V. Results

A. Design A: detection and escapes

The golden model predicts that March X detects 2,528 of the 4,032 faults; the DUT raises FAIL in 2,456 (Table II, Fig. 3). All 72 differences are escapes, a faulty memory that passes (2.85%, 95% CI 2.27–3.57%); there are no false fails. Every escape sits at address 0 or 31 (Fig. 4), the last address of the $\Downarrow (r1, w0)$ element and of the final $\Uparrow (r0)$ element.

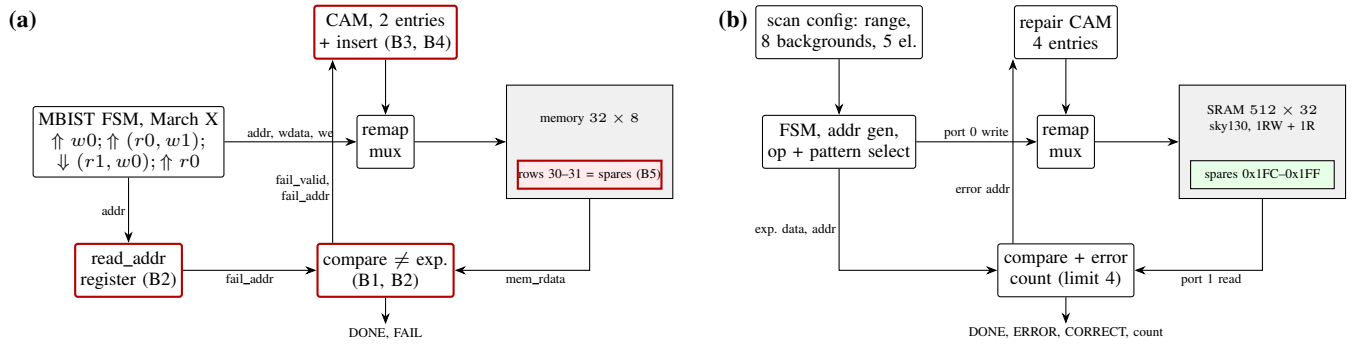


Fig. 2. Architecture of the two designs under test. (a) Design A (TT #777): red blocks are where bugs B1–B5 of Table III sit; the spare rows lie inside the tested range. (b) Design B (mbist_ctrl): programmable algorithm and data backgrounds, separate write and read ports, spares outside the tested range.

TABLE II
FAULT CAMPAIGN ON DESIGN A (SINGLE FAULTS).

Class	Faults	Golden det.	DUT FAIL	Escapes	Correct*
SAF	512	512	504	8	0
TF	512	512	496	16	0
CF inter-word	1,984	992	960	32	0
CF intra-word	1,024	512	496	16	0
Total	4,032	2,528	2,456	72	0

*Detected runs whose fail addresses and repair table both match the golden model.

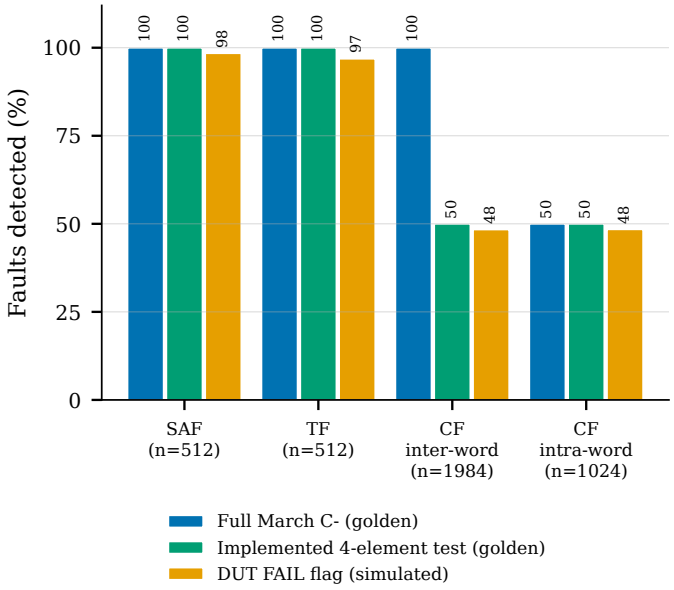


Fig. 3. Fault detection by class: full March C– and the implemented March X (golden model) versus Design A's FAIL flag.

B. What each checker level sees

Fig. 5 shows, for every detectable fault, whether its run exposes a bug. The design's own testbench (L0) passes every run. L1 exposes the 72 escapes. L2 exposes a bug in all 2,528 detectable faults (95% CI 99.85–100%): in every detected run the fail addresses and the repair table differ from the prediction. Of 2,712 fail reports that pair with a predicted

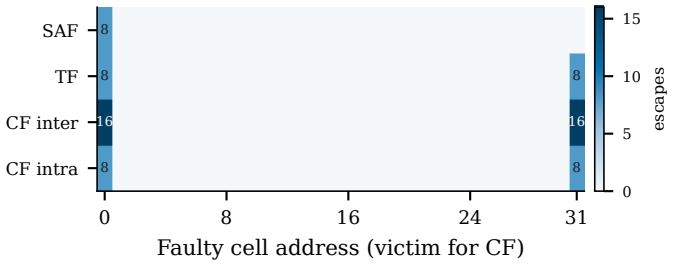


Fig. 4. Design A escapes by faulty-cell address and fault class. All 72 escapes are at addresses 0 and 31, the final addresses of the two descending/final read elements.

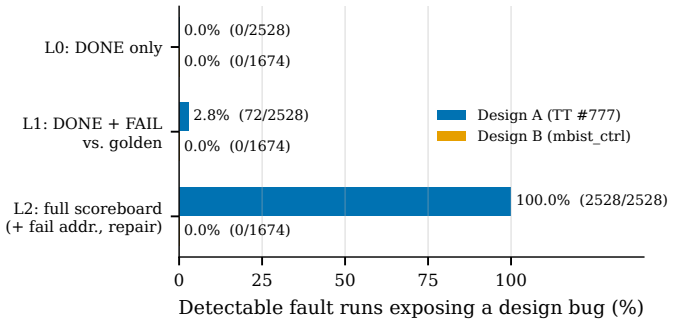


Fig. 5. Detectable fault runs that expose a design bug, by checker level, for both designs.

report, only 280 (10.3%) carry the correct address; 2,416 are off by one. The faulty cell reached the repair table in only 40 of 2,456 failing runs (1.63%, CI 1.20–2.21%), and in every failing run both repair entries held the same address.

C. Root causes

Table III lists the bugs, each reproduced in simulation and B1–B3 traced in waveforms. Fig. 6 shows the common cause of B1 and B2 at register level: the read data reaches the comparator two stages after its address, while the fail address comes from a register that advances one stage earlier. Fig. 7 shows B1 on the waveform: the read of address 0 returns FE, but the element has ended when the data arrives, so FAIL stays low.

TABLE III
DESIGN BUGS FOUND IN DESIGN A.

ID	Symptom	Root cause
B1	Escapes at addresses 0 and 31 (72 runs)	Last read of $\Downarrow$ $(r1, w0)$ and of final $\Uparrow$ $(r0)$ never compared
B2	Fail address off by one	Compare uses a read-address register that has advanced
B3	One failure fills both repair entries	Non-blocking "inserted" flag read in the same cycle
B4	Second faulty cell never recorded	Table already full of duplicates (B3); seen with two injected faults
B5	Spares overlap user space	Spare rows 30–31 are also tested user addresses
B6	Docs vs. RTL mismatch	256 × 8/16 entries documented; 32 × 8/2 implemented

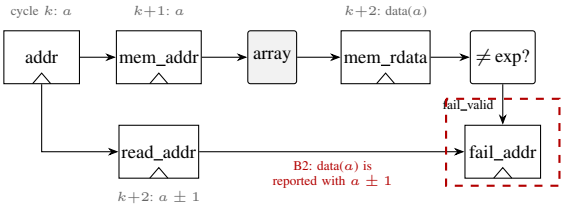


Fig. 6. Register-level view of Design A's read pipeline (boxes with a clock mark are registers). The read data reaches the comparator two stages after its address, but the fail address is taken from a register that has already advanced (B2). For the same reason the last read of an element is still in flight when the FSM leaves the element, so it is never compared (B1).

D. Design B: a control case

Table IV summarises Design B. Every single SAF and TF was detected, counted and repaired at the faulty word's address; sets of two to four faulty words were repaired exactly; sets of five or six were reported as unrepairable. No run exposed a bug at any level: 0 of 1,674 non-spare runs (95% upper bound 0.23%). Fig. 8 shows a repair: the read of word 16 returns 5555 5554 instead of 5555 5555, the word enters repair entry 0, and from the next access it is remapped to spare 0x1FC. A faulty spare that is never used has no effect; when it is used, the controller detects it, consumes the next entry for the same address, and reports ERROR only when the four entries run out. Code review found a latent off-by-one in the repair-counter bound ($\leq$ instead of $<$ the limit), unreachable because the comparator stops reporting at the limit.

E. Seeded bugs: qualifying the checkers

Fig. 9 shows the kill rate of the 34 seeded bugs. Completion-only checking kills 5 (15%; only bugs that change whether or when DONE asserts); the pass/fail flag kills 21 (62%); the full scoreboard kills 27 (79%). On Design A the step from L1 to L2 adds five bugs (50% to 81%), all in fail-address reporting and repair. On Design B, CORRECT and the count add one; repair errors there also change the count, so the repair table adds none. Table V lists the seven survivors: four do not change any output (equivalent), and three are masked by the test set, most notably "stops after the first data background", which only intra-word CFs would expose.

TABLE IV
FAULT CAMPAIGN ON DESIGN B.

Case	Runs	ERROR (golden / DUT)	L2 bugs
Single SAF (all words)	1,016	0 / 0	0
Single TF (every 2nd word)	508	0 / 0	0
2–4 faulty words	90	0 / 0	0
5–6 faulty words	60	60 / 60	0
Faulty spare + 0–4 faults	20	– / 4	–

TABLE V
SEEDED BUGS THAT SURVIVE ALL CHECKER LEVELS.

ID	Seeded change	Why it survives
A06	FAIL not set by first r0 element	Masked: same faults also fail later reads
A07	Fail address from `addr` not `read_addr`	Equivalent in this element
A14	Duplicate check disabled	Equivalent given B3
B08	Comparator CORRECT never set	Top-level CORRECT comes from repair block
B13	Remap also scans unwritten entry	Equivalent
B14	Stops after first background	Masked: needs intra-word CF
B15	No abort on unrepairable error	Same final outputs

F. Algorithm adequacy

Independently of the RTL, the golden model compares algorithms on the same fault list (Fig. 10). Design A's March X detects 50% of inter-word CFs where March C– detects 100%; with solid data both detect 50% of intra-word CFs. Design B's default algorithm and March C– with four backgrounds detect all four classes.

VI. DISCUSSION

A. Recommendations for MBIST/BISR Designers

The bugs suggest rules that are cheap in new designs: (1) carry the address with the read data through the pipeline and report the address that travelled with the failing data, not a controller register; (2) drain every outstanding read before leaving a March element or the test; (3) make repair insertion a one-hot priority write with a same-cycle duplicate check; (4) keep spare resources outside the tested address space, or test them first; (5) expose the fail-address stream and repair registers to the testbench, since a pass/fail flag hides most bugs; and (6) qualify the testbench with seeded bugs, as completion-only checks kill fewer than one in five.

B. Limitations and Open Questions

The study is simulation-only and uses small open-source memories; industrial MBIST adds programmable algorithms, shared controllers and physical address scrambling. Faults are behavioural; for Design A they are single faults, so multi-fault repair (B4) was examined with directed tests, while Design B

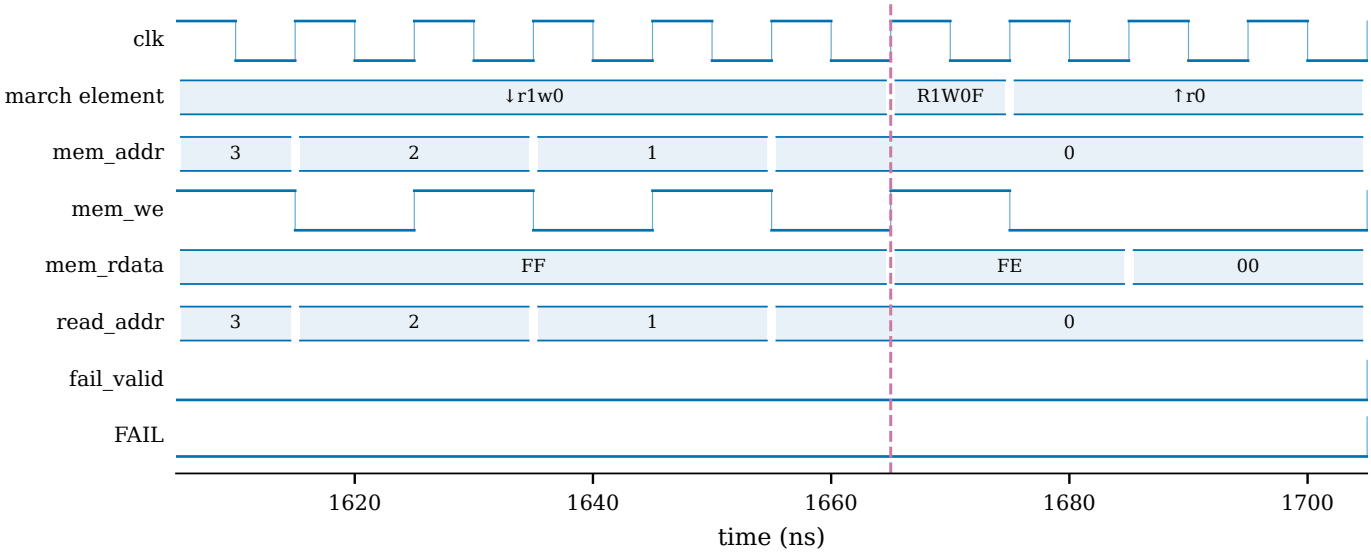


Fig. 7. B1 on Design A: stuck-at-0 at address 0. The faulty read data (FE) arrives after $\Downarrow (r1, w0)$ ends (dashed line) and is never compared; fail_valid never rises and FAIL stays low.

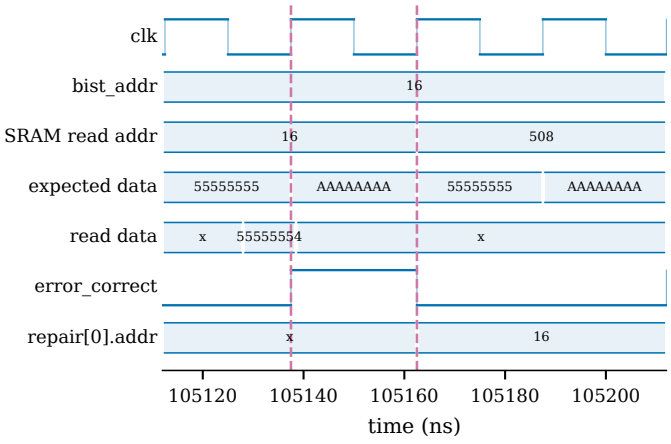


Fig. 8. Design B: stuck-at-0 in word 16 is detected (read 55555554, expected 55555555), entered in repair entry 0, and remapped to spare word 0x1FC (508) from the next access.

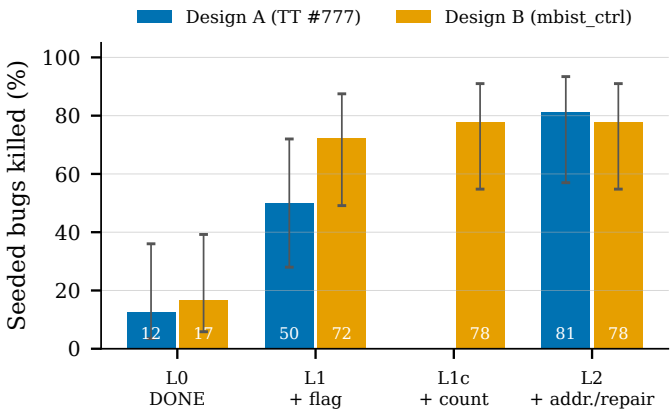


Fig. 9. Seeded RTL bugs killed per checker level (error bars: Wilson 95% CI). L1c applies to Design B only.

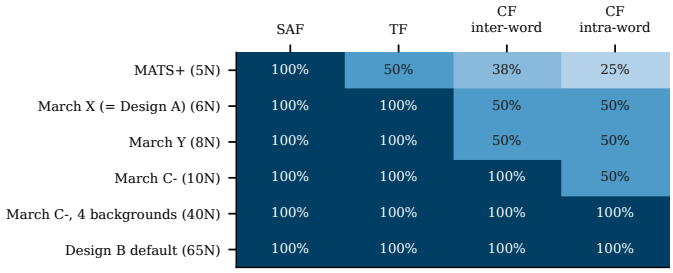


Fig. 10. Golden-model coverage of March algorithms on a 32×8 memory (same fault list as Table II); operations per word in parentheses.

was run with up to six faulty words. The golden models encode an interpretation of each design's intent (spares outside the tested range, repair of the first R distinct addresses). Design B's seeded-bug test set is small (seven tests); adding intra-word CFs would expose B14. Open questions are a chip-level run with several MBIST instances behind one bus and formal equivalence between the golden model and the controller's algorithm encoding.

VII. Conclusion

Checking only completion and the pass/fail flag gives false confidence in MBIST and repair logic. On a taped-out open-source design, a golden-model scoreboard exposed escapes, wrong fail addresses and a broken repair table in every detectable fault, none of which the design's own testbench detected, and raised no false alarm on a second, well-built design. Seeded bugs confirm that each added checker level kills bugs the weaker ones miss. Designs, testbenches, scripts and results are available at https://github.com/abhinavkidambi1989/p0-mbist-verif (public upon publication).

ACKNOWLEDGMENT

The author used an AI assistant (Claude, Anthropic) to help draft text, testbench code, scripts and figures. All content was reviewed, verified and revised by the author, who takes full responsibility for it.

REFERENCES

[1] Y. Zorian and S. Shoukourian, "Embedded-memory test and repair: Infrastructure IP for SoC yield," *IEEE Design & Test of Computers*, vol. 20, no. 3, pp. 58–66, 2003.

[2] A. J. van de Goor, *Testing Semiconductor Memories: Theory and Practice*. Chichester, U.K.: Wiley, 1991.

[3] A. J. van de Goor, "Using March tests to test SRAMs," *IEEE Design & Test of Computers*, vol. 10, no. 1, pp. 8–14, 1993.

[4] J. Knaizuk and C. R. P. Hartmann, "An optimal algorithm for testing stuck-at faults in random access memories," *IEEE Trans. Computers*, vol. C-26, no. 11, pp. 1141–1144, 1977.

[5] M. Marinescu, "Simple and efficient algorithms for functional RAM testing," in *Proc. IEEE Int. Test Conf. (ITC)*, 1982, pp. 236–239.

[6] A. J. van de Goor, G. N. Gaydadjiev, V. N. Yarmolik, and V. G. Mikitjuk, "March LR: A test for realistic linked faults," in *Proc. IEEE VLSI Test Symp.*, 1996.

[7] S. M. Al-Harbi and S. K. Gupta, "Generating complete and optimal March tests for linked faults in memories," in *Proc. IEEE VLSI Test Symp.*, 2003.

[8] A. J. van de Goor and B. Smit, "Automating the verification of memory tests," in *Proc. IEEE VLSI Test Symp.*, 1994, pp. 312–318.

[9] A. Kumar, "Assertion based functional verification of March algorithm based MBIST controller," arXiv:2106.11461, 2021.

[10] S. Tulsyan, V. Pillai A, M. Jalasutram, P. Sinha, and M. Parasrampuria, "Physical design aware verification methodology for closing coverage gaps in SharedBus MBIST," IEEE, 2026, IEEE Xplore doc. 11646504.

[11] V. Sontakke and D. Atchina, "Memory built-in self-repair and correction for improving yield: A review," *Int. J. Electrical and Computer Engineering*, vol. 14, no. 1, pp. 140–156, 2024.

[12] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[13] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[14] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[15] Tiny Tapeout, "IHP 26a project #777: MBIST with MBISR," https://tinytapeout.com/chips/ttihp26a/777, accessed Nov. 13, 2026.

[16] D. Annayya, "mbist_ctrl," https://github.com/dineshannayya/mbist_ctrl, accessed Nov. 13, 2026.

[17] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog.